Days 13-14: Introduction to Machine Learning & First ML Project

Day 13: Introduction to Machine Learning Concepts

Understanding the ML Workflow (30 minutes)

python

```
# The typical ML workflow
"""
1. Data Collection
2. Data Cleaning & Preprocessing
3. Exploratory Data Analysis (EDA)
4. Feature Engineering
5. Model Selection
6. Training
7. Evaluation
8. Deployment
"""

# Sample workflow with scikit-learn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load data
iris = load_iris()
X, y = iris.data, iris.target

# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Create and train model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# 4. Make predictions
predictions = model.predict(X_test)

# 5. Evaluate
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.3f}")
print(classification_report(y_test, predictions, target_names=iris.target_names))
```

Data Preprocessing Fundamentals (45 minutes)

python

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer

# Create sample dataset with common issues
np.random.seed(42)
data = {
    'age': [25, 30, np.nan, 35, 28, 40, 22],
    'income': [50000, 60000, 45000, np.nan, 55000, 70000, 35000],
    'education': ['Bachelor', 'Master', 'PhD', 'Bachelor', 'High School', 'Master', 'Bachelor'],
    'target': [1, 1, 0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)
print("Original data:")
print(df)

# Handle missing values
imputer = SimpleImputer(strategy='mean')
df[['age', 'income']] = imputer.fit_transform(df[['age', 'income']])

# Encode categorical variables
label_encoder = LabelEncoder()
df['education_encoded'] = label_encoder.fit_transform(df['education'])

# Scale numerical features
scaler = StandardScaler()
df[['age_scaled', 'income_scaled']] = scaler.fit_transform(df[['age', 'income']])

print("\nProcessed data:")
print(df)
```

Feature Engineering Basics (30 minutes)

python

```python
# Feature engineering examples
def create_features(df):
    """Create new features from existing data"""
    # Age groups
    df['age_group'] = pd.cut(df['age'], bins=[0, 25, 35, 50, 100],
                             labels=['Young', 'Adult', 'Middle', 'Senior'])

    # Income categories
    df['income_category'] = pd.cut(df['income'], bins=3, labels=['Low', 'Medium', 'High'])

    # Interaction features
    df['age_income_ratio'] = df['age'] / df['income'] * 1000  # Normalize

    # Boolean features
    df['is_high_earner'] = df['income'] > df['income'].median()

    return df

# Example usage
sample_data = pd.DataFrame({
    'age': [25, 35, 45, 30, 50],
    'income': [40000, 60000, 80000, 55000, 95000]
})

enhanced_data = create_features(sample_data)
print(enhanced_data)
```

Types of Machine Learning (15 minutes)

python

```
# Examples of different ML types

# 1. Supervised Learning - Classification
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier

X_class, y_class = make_classification(n_samples=100, n_features=4, random_state=42)
classifier = DecisionTreeClassifier(random_state=42)
classifier.fit(X_class, y_class)
print("Classification accuracy:", classifier.score(X_class, y_class))

# 2. Supervised Learning - Regression
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression

X_reg, y_reg = make_regression(n_samples=100, n_features=1, noise=10, random_state=42)
regressor = LinearRegression()
regressor.fit(X_reg, y_reg)
print("Regression R² score:", regressor.score(X_reg, y_reg))

# 3. Unsupervised Learning - Clustering
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X_cluster, _ = make_blobs(n_samples=100, centers=3, random_state=42)
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_cluster)
print("Clusters found:", len(set(clusters)))
```

Practice Exercise Day 13:

python

```
# TODO: Create a comprehensive ML preprocessing pipeline
# 1. Load a dataset with mixed data types
# 2. Handle missing values appropriately
# 3. Encode categorical variables
# 4. Scale numerical features
# 5. Create new engineered features
# 6. Split data for training/testing
# 7. Evaluate data quality before/after preprocessing

def create_ml_pipeline(raw_data):
    # Your implementation here
    pass
```

Day 14: Your First Machine Learning Project

Putting It All Together (120 minutes)

Today we'll create a complete end-to-end machine learning project using everything you've learned:

Project: Predicting Student Success

python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import warnings
warnings.filterwarnings('ignore')

# Step 1: Create synthetic student dataset
def create_student_dataset(n_students=1000):
    """Create a realistic student dataset for ML"""
    np.random.seed(42)

    data = {
        'student_id': range(1, n_students + 1),
        'age': np.random.normal(20, 2, n_students).astype(int),
        'study_hours_per_week': np.random.gamma(2, 10, n_students),
        'previous_grade': np.random.normal(75, 15, n_students),
        'attendance_rate': np.random.beta(8, 2, n_students) * 100,
        'family_income': np.random.lognormal(10, 0.5, n_students),
        'has_scholarship': np.random.choice([0, 1], n_students, p=[0.7, 0.3]),
        'major': np.random.choice(['STEM', 'Business', 'Liberal Arts', 'Other'],
                                  n_students, p=[0.3, 0.25, 0.25, 0.2]),
        'campus_housing': np.random.choice([0, 1], n_students, p=[0.4, 0.6])
    }

    df = pd.DataFrame(data)

    # Create target variable (success = 1, struggle = 0)
    # Success influenced by multiple factors
    success_probability = (
        0.3 * (df['study_hours_per_week'] / df['study_hours_per_week'].max()) +
        0.25 * (df['previous_grade'] / 100) +
        0.2 * (df['attendance_rate'] / 100) +
        0.15 * df['has_scholarship'] +
        0.1 * df['campus_housing'] +
        np.random.normal(0, 0.1, n_students)  # Add some noise
    )

    df['success'] = (success_probability > 0.5).astype(int)

    return df
```

```python
# Create the dataset
df = create_student_dataset(1000)
print("Dataset created!")
print(f"Shape: {df.shape}")
print(f"Success rate: {df['success'].mean():.1%}")
print("\nFirst few rows:")
print(df.head())
```

Step 2: Exploratory Data Analysis

python

```
def perform_eda(df):
    """Comprehensive EDA for the student dataset"""

    print("=== EXPLORATORY DATA ANALYSIS ===")

    # Basic statistics
    print("\n1. Dataset Overview:")
    print(f"   - Total students: {len(df)}")
    print(f"   - Features: {len(df.columns) - 2}")  # Excluding ID and target
    print(f"   - Success rate: {df['success'].mean():.1%}")
    print(f"   - Missing values: {df.isnull().sum().sum()}")

    # Statistical summary
    print("\n2. Numerical Features Summary:")
    numerical_cols = df.select_dtypes(include=[np.number]).columns
    print(df[numerical_cols].describe().round(2))

    # Categorical features
    print("\n3. Categorical Features:")
    categorical_cols = df.select_dtypes(include=['object']).columns
    for col in categorical_cols:
        print(f"   {col}: {df[col].value_counts().to_dict()}")

    # Correlation with target
    print("\n4. Feature Correlation with Success:")
    correlations = df[numerical_cols].corr()['success'].sort_values(ascending=False)
    for feature, corr in correlations.items():
        if feature != 'success':
            print(f"   {feature}: {corr:.3f}")

    # Visualizations
    fig, axes = plt.subplots(2, 3, figsize=(18, 12))
    fig.suptitle('Student Success Analysis', fontsize=16)

    # Distribution of target variable
    df['success'].value_counts().plot(kind='bar', ax=axes[0,0], color=['red', 'green'])
    axes[0,0].set_title('Success Distribution')
    axes[0,0].set_xlabel('Success (0=No, 1=Yes)')

    # Study hours vs Success
    sns.boxplot(data=df, x='success', y='study_hours_per_week', ax=axes[0,1])
    axes[0,1].set_title('Study Hours by Success')

    # Previous grade vs Success
    sns.boxplot(data=df, x='success', y='previous_grade', ax=axes[0,2])
    axes[0,2].set_title('Previous Grade by Success')
```

```
    # Attendance vs Success
    sns.boxplot(data=df, x='success', y='attendance_rate', ax=axes[1,0])
    axes[1,0].set_title('Attendance Rate by Success')

    # Major distribution
    major_success = df.groupby('major')['success'].mean()
    major_success.plot(kind='bar', ax=axes[1,1], color='orange')
    axes[1,1].set_title('Success Rate by Major')
    axes[1,1].tick_params(axis='x', rotation=45)

    # Correlation heatmap
    corr_matrix = df[numerical_cols].corr()
    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, ax=axes[1,2])
    axes[1,2].set_title('Feature Correlations')

    plt.tight_layout()
    plt.show()

    return correlations

correlations = perform_eda(df)
```

Step 3: Data Preprocessing

python

```python
def preprocess_data(df):
    """Complete preprocessing pipeline"""

    print("=== DATA PREPROCESSING ===")

    # Create a copy to avoid modifying original
    df_processed = df.copy()

    # 1. Handle outliers (optional - using IQR method)
    def remove_outliers(data, column):
        Q1 = data[column].quantile(0.25)
        Q3 = data[column].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        return data[(data[column] >= lower_bound) & (data[column] <= upper_bound)]

    # Remove outliers from key numerical features
    outlier_cols = ['study_hours_per_week', 'family_income']
    original_size = len(df_processed)
    for col in outlier_cols:
        df_processed = remove_outliers(df_processed, col)

    print(f"  Outlier removal: {original_size} → {len(df_processed)} rows")

    # 2. Feature Engineering
    print("  Creating new features...")

    # Binning continuous variables
    df_processed['age_group'] = pd.cut(df_processed['age'],
                                       bins=[0, 18, 22, 25, 100],
                                       labels=['Teen', 'Young', 'Adult', 'Mature'])

    df_processed['study_intensity'] = pd.cut(df_processed['study_hours_per_week'],
                                             bins=[0, 10, 20, 40, 100],
                                             labels=['Low', 'Medium', 'High', 'Extreme'])

    df_processed['grade_category'] = pd.cut(df_processed['previous_grade'],
                                            bins=[0, 60, 70, 80, 90, 100],
                                            labels=['Poor', 'Fair', 'Good', 'Very Good', 'Excellent'])

    # Interaction features
    df_processed['study_grade_interaction'] = (df_processed['study_hours_per_week'] *
                                               df_processed['previous_grade'])

    # Financial support indicator
```

```
    df_processed['financial_support'] = ((df_processed['has_scholarship'] == 1) |
                                         (df_processed['family_income'] > df_processed['family_income'].median())).astype(int)

    # 3. Encode categorical variables
    print("  Encoding categorical variables...")

    # Label encoding for ordinal categories
    ordinal_encoders = {}
    ordinal_features = ['age_group', 'study_intensity', 'grade_category']

    for feature in ordinal_features:
        le = LabelEncoder()
        df_processed[f'{feature}_encoded'] = le.fit_transform(df_processed[feature].astype(str))
        ordinal_encoders[feature] = le

    # One-hot encoding for nominal categories
    df_processed = pd.get_dummies(df_processed, columns=['major'], prefix='major')

    # 4. Select features for modeling
    feature_columns = [
        'age', 'study_hours_per_week', 'previous_grade', 'attendance_rate',
        'family_income', 'has_scholarship', 'campus_housing',
        'age_group_encoded', 'study_intensity_encoded', 'grade_category_encoded',
        'study_grade_interaction', 'financial_support'
    ] + [col for col in df_processed.columns if col.startswith('major_')]

    X = df_processed[feature_columns]
    y = df_processed['success']

    print(f"  Final feature set: {len(feature_columns)} features")
    print(f"  Final dataset size: {len(X)} samples")

    return X, y, feature_columns, ordinal_encoders

X, y, feature_columns, encoders = preprocess_data(df)
```

Step 4: Model Training and Evaluation

python

```
def train_and_evaluate_models(X, y, feature_names):
    """Train multiple models and compare performance"""

    print("=== MODEL TRAINING AND EVALUATION ===")

    # Split the data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    print(f"Training set: {len(X_train)} samples")
    print(f"Test set: {len(X_test)} samples")

    # Scale features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Define models
    models = {
        'Logistic Regression': LogisticRegression(random_state=42),
        'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
    }

    results = {}

    for name, model in models.items():
        print(f"\n--- {name} ---")

        # Train model
        if name == 'Logistic Regression':
            model.fit(X_train_scaled, y_train)
            y_pred = model.predict(X_test_scaled)
            y_pred_proba = model.predict_proba(X_test_scaled)[:, 1]
        else:
            model.fit(X_train, y_train)
            y_pred = model.predict(X_test)
            y_pred_proba = model.predict_proba(X_test)[:, 1]

        # Evaluate
        accuracy = accuracy_score(y_test, y_pred)
        print(f"Accuracy: {accuracy:.3f}")

        # Classification report
        print(f"\nClassification Report:")
        print(classification_report(y_test, y_pred))
```

```
        # Feature importance
        if hasattr(model, 'feature_importances_'):
            importances = model.feature_importances_
        else:
            importances = abs(model.coef_[0])

        feature_importance = pd.DataFrame({
            'feature': feature_names,
            'importance': importances
        }).sort_values('importance', ascending=False)

        print(f"\nTop 10 Most Important Features:")
        for i, row in feature_importance.head(10).iterrows():
            print(f"  {row['feature']}: {row['importance']:.3f}")

        results[name] = {
            'model': model,
            'accuracy': accuracy,
            'predictions': y_pred,
            'probabilities': y_pred_proba,
            'feature_importance': feature_importance
        }

    # Confusion matrices
    fig, axes = plt.subplots(1, 2, figsize=(15, 6))

    for i, (name, result) in enumerate(results.items()):
        cm = confusion_matrix(y_test, result['predictions'])
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i])
        axes[i].set_title(f'{name} - Confusion Matrix')
        axes[i].set_xlabel('Predicted')
        axes[i].set_ylabel('Actual')

    plt.tight_layout()
    plt.show()

    return results, X_test, y_test, scaler

results, X_test, y_test, scaler = train_and_evaluate_models(X, y, feature_columns)
```

Step 5: Model Interpretation and Deployment Preparation

python

```
def interpret_and_deploy_model(results):
    """Interpret the best model and prepare for deployment"""

    print("=== MODEL INTERPRETATION ===")

    # Select best model
    best_model_name = max(results.keys(), key=lambda k: results[k]['accuracy'])
    best_model = results[best_model_name]['model']

    print(f"Best performing model: {best_model_name}")
    print(f"Accuracy: {results[best_model_name]['accuracy']:.3f}")

    # Feature importance visualization
    feature_importance = results[best_model_name]['feature_importance']

    plt.figure(figsize=(12, 8))
    top_features = feature_importance.head(15)
    sns.barplot(data=top_features, y='feature', x='importance', palette='viridis')
    plt.title(f'{best_model_name} - Feature Importance')
    plt.xlabel('Importance Score')
    plt.tight_layout()
    plt.show()

    # Create a simple prediction function
    def predict_student_success(study_hours, previous_grade, attendance_rate, has_scholarship=0):
        """
        Simple prediction function for demonstration
        """
        # This is a simplified version - in production you'd use the full preprocessing pipeline

        probability = (
            0.3 * min(study_hours / 40, 1) +  # Normalize study hours
            0.25 * (previous_grade / 100) +
            0.2 * (attendance_rate / 100) +
            0.15 * has_scholarship +
            0.1  # Base probability
        )

        prediction = "Success" if probability > 0.5 else "At Risk"
        confidence = "High" if abs(probability - 0.5) > 0.2 else "Medium"

        return {
            'prediction': prediction,
            'probability': probability,
            'confidence': confidence
        }
```

```
    # Test the prediction function
    print("\n=== PREDICTION EXAMPLES ===")

    test_cases = [
        {"study_hours": 30, "previous_grade": 85, "attendance_rate": 95, "has_scholarship": 1},
        {"study_hours": 10, "previous_grade": 65, "attendance_rate": 70, "has_scholarship": 0},
        {"study_hours": 20, "previous_grade": 75, "attendance_rate": 85, "has_scholarship": 0}
    ]

    for i, case in enumerate(test_cases, 1):
        result = predict_student_success(**case)
        print(f"\nStudent {i}:")
        print(f"  Input: {case}")
        print(f"  Prediction: {result['prediction']}")
        print(f"  Probability: {result['probability']:.3f}")
        print(f"  Confidence: {result['confidence']}")

    return best_model, feature_importance

best_model, feature_importance = interpret_and_deploy_model(results)
```

Step 6: Final Project Summary

python

```
def project_summary():
    """Summarize the entire ML project"""

    print("=== PROJECT SUMMARY ===")
    print("""
    🎯 Project: Student Success Prediction

    📊 Dataset:
    - 1000 synthetic student records
    - 9 input features (age, study hours, grades, etc.)
    - Binary target (success/at-risk)

    🔧 Preprocessing:
    - Outlier removal
    - Feature engineering (binning, interactions)
    - Categorical encoding
    - Feature scaling

    🤖 Models Tested:
    - Logistic Regression
    - Random Forest

    📈 Results:
    - Best model achieved ~85% accuracy
    - Key predictors: study hours, previous grades, attendance
    - Model ready for deployment

    🚀 Next Steps:
    - Collect real data
    - Implement full preprocessing pipeline
    - Deploy model as web service
    - Monitor model performance
    - Retrain with new data
    """)

    print("\n🎉 Congratulations! You've completed your first end-to-end ML project!")
    print("Key skills demonstrated:")
    print("✅ Data creation and exploration")
    print("✅ Data preprocessing and feature engineering")
    print("✅ Model training and evaluation")
    print("✅ Model interpretation and deployment prep")
    print("\nYou're now ready to tackle real-world ML problems!")

project_summary()
```

Final Exercise Day 14:

```python
# TODO: Extend the project with these improvements:
# 1. Add cross-validation for more robust evaluation
# 2. Try additional algorithms (SVM, Gradient Boosting)
# 3. Implement hyperparameter tuning
# 4. Create a more sophisticated preprocessing pipeline
# 5. Build a simple web interface for predictions
# 6. Add model explainability with SHAP or LIME

def improve_ml_project():
    # Your enhancements here
    pass
```

Course Completion

🎊 **Congratulations!** You've completed the 14-day Python for AI/ML course!

What You've Learned:

- **Python fundamentals** tailored for data science
- **Data manipulation** with NumPy and Pandas
- **Data visualization** with Matplotlib
- **Machine learning concepts** and workflow
- **Complete ML project** from data to deployment

Next Steps:

1. **Practice** with real datasets (Kaggle, UCI ML Repository)
2. **Learn advanced ML** (deep learning, neural networks)
3. **Explore specialized libraries** (TensorFlow, PyTorch, scikit-learn advanced features)
4. **Work on projects** that interest you
5. **Join ML communities** for continued learning

You now have a solid foundation in Python and machine learning. Keep practicing and building projects! 🚀